

Development of a Web-Based Real-Time Robotic Control System Using ESP32 and Cloud Technologies

AYUSH GUPTA¹ and AFTAB AHMED ANSARI^{2*}

^{1,2} Centre for Internet of Things, Madhav Institute of Technology and Science, Gwalior,
Madhya Pradesh, India

22e010ay622@mitsgwl.ac.in

*aftabansari@mitsgwalior.in

Abstract. Internet-controlled robotics is an emerging application of Internet of Things (IoT) technologies that enables remote operation of robotic systems through cloud-based architecture. This paper presents the design and implementation of an ESP32-based robotic control system that can be operated through the internet using a cloud-based architecture. The system integrates an ESP32 microcontroller with an L298N motor driver to control a mobile robot, while communication between the robot and the user interface is achieved through two ways first one is the Firebase Realtime Database and the other one is using the HiveMQ broker. A web interface hosted on GitHub Pages/Netlify allows users to send movement commands which are transmitted to the Firebase Realtime Database or HiveMQ broker and then delivered to the ESP32 which continuously listens for command updates through a persistent connection from the Firebase Realtime Database or HiveMQ broker & executes corresponding motor actions such as forward, backward, left, and right movements, enabling real-time communication between the user and the robot. Experimental evaluations of the system show an average command response latency of roughly 350ms. The proposed architecture presents a low-cost and scalable approach for real-time, internet-controlled remote robotics using ESP32, cloud services, and web technologies.

Keywords: IoT, Internet, Web Technologies, Cloud, Database, Persistent Connection, Firebase Realtime Database, HiveMQ

1 Introduction

Using Internet of Things (IoT) devices can interact and controlled through internet. One main application of IoT is remote robotics, where robots can be monitored and controlled from distant locations using cloud-based architecture. In IoT-based remote robotics, sensors & microcontrollers connects to Internet through communication modules such Wi-fi or GSM allows robots to exchange data with cloud platforms such as Firebase, HiveMQ & user interfaces in real time.

In a typical IoT remote robotic system, a microcontroller such as the ESP32 connects to the internet using wireless communication technologies like Wi-Fi or cellular networks. User commands from web interface are transmitted through cloud services such as Firebase, MQTT brokers, or other IoT platforms. These commands are then received by the robot and translated into physical actions such as movement, sensing,

or task execution. In addition to receiving commands, the robot can also transmit operational data back to the cloud which enables a closed-loop feedback system, where the cloud or user interface can verify whether the command sent by the user has been successfully executed by the robot.

IoT-based remote robotics has many real-world applications in various fields. It can be used for security and surveillance in sensitive areas like borders, military zones etc.. In disaster situations, rescue robots used to explore dangerous environments where humans cannot go safely. Industries use such robots to improve automation and efficiency in manufacturing processes. In smart agriculture, these robots used for monitoring crops & environmental conditions, and perform tasks such as irrigation & pest detection.

2 Proposed System Architecture

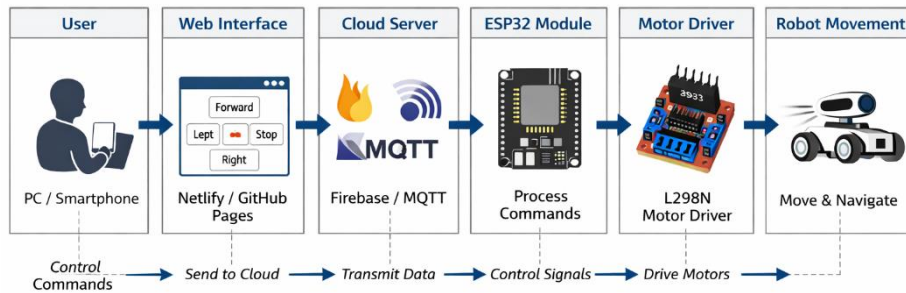


Fig. 1. Overall System Flow Architecture

The system enables users to remotely control a mobile robot through a cloud-based communication architecture. As shown in the Fig. 1., the process begins with the user, who sends control commands using buttons such as forward, backward, left, right, and stop of a web interface which has persistent connection with cloud accessible through any internet connected device, which is hosted on Netlify or GitHub Pages, When a button is pressed or released respective command is send to the cloud server, where communication is handled using either Firebase Realtime Database or the MQTT protocol with the HiveMQ broker. The cloud service acts as an intermediary that stores and forwards control commands to ESP32 through a persistent connection.

Using persistent connection with the cloud ESP32 retrieves these control commands in real time. After processing the received command, the ESP32 generates appropriate control signals for the L298N motor driver, which drives the robot's DC motors. As a result, the robot executes the corresponding movements.

2.1 System Hardware Components

ESP32 Microcontroller

The ESP32 is a low-cost, high-performance microcontroller with low power consumption widely used in IoT and embedded systems. It integrates Wi-Fi & Bluetooth, enabling communication with cloud platforms & remote applications over internet.

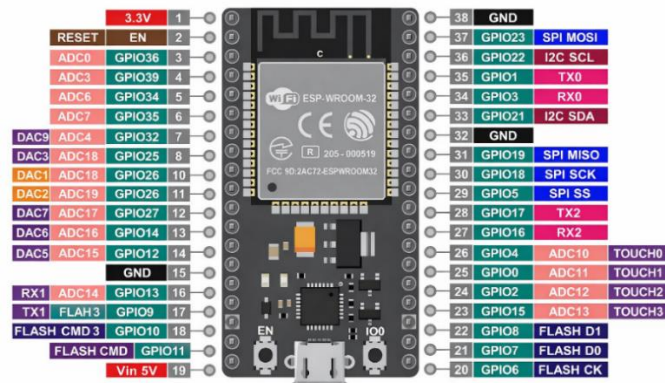


Fig. 2. ESP32 with Pinout Diagram

L298 Motor Driver

The L298N is a dual full H-Bridge motor driver IC used to control the direction & speed of up to two DC motors simultaneously & independently. This module contains L298N chip, which has two independent H-bridge circuits. Each H-bridge allows the direction of a motor to be controlled electronically.

The module also supports Pulse Width Modulation (PWM), which allows precise control of the motor speed by varying the duty cycle of the input signal by rapidly switching voltage ON and OFF. The duty cycle determines the motor speed.

- Higher duty cycle → Higher average voltage → Faster motor
- Lower duty cycle → Lower average voltage → Slower motor

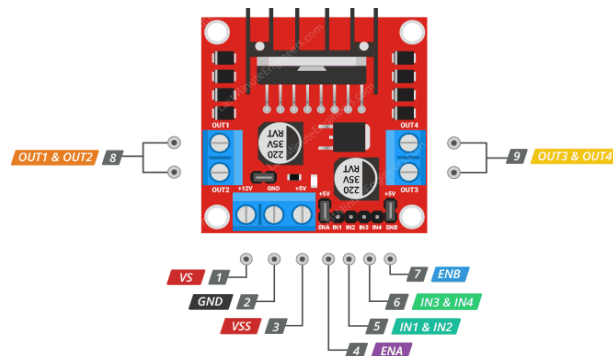


Fig. 3. L298 Motor Driver with Pinout Diagram

300 RPM Straight Single Shaft BO Motor

It is a low-cost geared DC motor widely used in small robotic applications which provides a good balance of torque and rotational speed at low operating voltages. The motor operates within a voltage range of 3–12 V. And have mounting holes for easy installation in robotic chassis.



Fig. 4. 300 RPM straight single-shaft BO motor

Metal Chassis, 65 mm Robot Wheel for BO Motors & Caster Wheel

The metal chassis is made up of metal which provides stability & structural support for components with pre-drilled mounting holes allows easy installation.

The 65 mm robot wheel is made up of Reinforced nylon hub for strong and durable structure with Rubber tire surface providing better grip & anti-slip performance, designed for BO motors. It converts motor rotation into linear motion.

A caster wheel consists of a ball bearing mechanism that allows 360° smooth rotation in all directions, improving robot mobility and reducing drag during movement.



Fig. 5. Red Metal Chassis, 65mm Wheel & Caster wheel

3.7V 1200mAh 18650 Battery, Battery Holder & Connecting Wires

The 3.7V 1200mAh 18650 is a rechargeable lithium-ion battery which provides stable voltage & long battery life making it suitable for powering IoT devices.



Fig. 6. 3.7V 1200mAh 18650 Battery, Battery Holder & Connecting Wires

2.2 System Software Components

Web Interface

A web interface is the user interaction layer of a system that allows users to interact with app such as send inputs and view responses from the system through a web browser. It is developed using HTML, CSS, JavaScript or related frameworks which provide visual elements such as buttons, forms, and layouts for user interaction. It can be deployed on cloud-based hosting platforms such as GitHub Pages or Netlify, which is accessible through a public URL, from any internet-connected device.

Firestore Realtime Database

It is a NoSQL cloud-hosted database which stores data as JSON. Instead of typical HTTP requests, the it uses real-time data synchronization every time data changes, any connected device receives that update within milliseconds.

HiveMQ MQTT Broker

HiveMQ cloud is a fully-managed, cloud-native MQTT-based messaging IoT platform that makes scalable IoT device connectivity simple. Its architecture ensures fast, reliable, secure and real-time movement of data to and from connected IoT devices.

It uses the MQTT protocol, which follows a publish-subscribe communication model where devices called publishers send messages to specific topics on MQTT broker, while other devices called subscribers receive those messages by subscribing to the same topics. It acts as an intermediary that manages message delivery between publishers and subscribers.

3 System Implementation

This section describes the hardware and software implementation of the proposed internet-controlled robotic system.

3.1 Software Implementation

Web Interface with Firebase Realtime Database

The web interface & ESP32(using the Firebase ESP Client library) both establishes an authenticated persistent connection (real-time data stream) with Firebase Realtime Database using Firebase JavaScript SDK, using the configuration credentials provided in Project Settings → General → Your Apps → Web App, with the user credentials created in the Authentication section under Security. The configuration includes parameters such as the API key, database URL, email, and password etc.. Using the established persistent connection, the web interface can send control commands securely to the Firebase Realtime Database & the ESP32 receive these commands.

Web Interface with HiveMQ MQTT Broker

The web interface(using the MQTT JavaScript client library) & ESP32(using the PubSubClient Client library) both establishes an authenticated persistent connection with the HiveMQ MQTT broker. The connection is authenticated using the user's username & password created in the HiveMQ dashboard, which serve as broker authentication credentials. Using the established persistent MQTT connection, the web interface publishes control commands to a specific MQTT topic, and subscribed client which is ESP32 receive these messages instantly allowing the broker to deliver messages instantly whenever a command is published.

When user presses or releases a button on web interface then it sends the corresponding control commands to cloud in (direction, speed) format where direction's value can be 1, 2, 3, 4 & 5 which corresponds to Forward, Backward, Left & Right buttons of web interface.



Fig. 7. Web Interface

The speed value ranges from 0-255, which corresponds to the 8-bit PWM resolution of the ESP32 which provides 256 discrete levels for controlling the motor speed.

To increase the speed in a particular direction, the corresponding button must be held down continuously. When the button is pressed, an initial speed value of 80 is transmitted to the cloud with the corresponding direction. If button remains pressed, the speed value increases by 20 units per second, similar to the acceleration mechanism in a vehicle. The speed change is implemented for forward & backward direction. For left & right direction speed value 70 unit is fixed.

After pressing the button, when the button is released, the command (0, 0) is immediately sent to the server, which is stop the motors.

Firestore Realtime Database Communication

In the Firestore implementation, the web interface sends control commands to a specific node in the Firestore Realtime Database through an authenticated persistent connection. When the database value changes at the specified node, the Firestore Realtime Database automatically pushes updates through the active persistent connection, which are then received by the ESP32 through the stream callback function. The received command contains the direction and motor speed values, which are parsed by the ESP32 and used to control the motor driver.

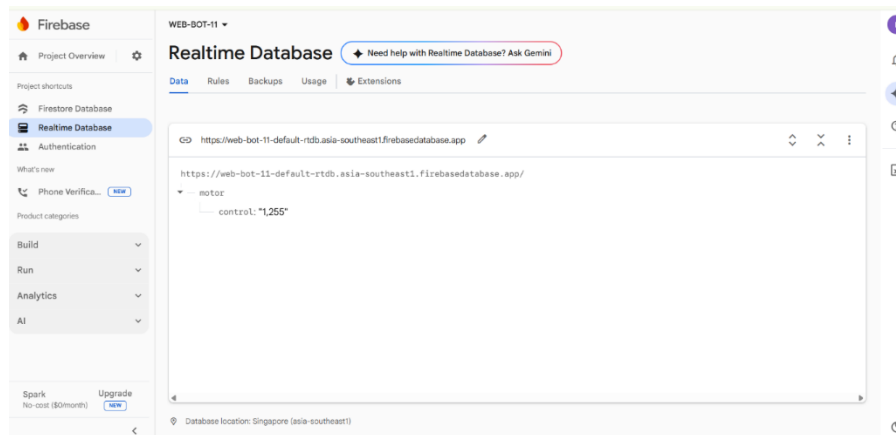


Fig. 8. Firestore Realtime Database Interface where data sent to control node

MQTT Communication

In this implementation, the web interface publishes control messages to a specific MQTT topic. The ESP32 connects securely to the HiveMQ broker using internet and subscribes to the control topic. When a message is received, the ESP32 parses the command and executes the corresponding motor control function.

3.2 Hardware Implementation

ESP32 Microcontroller

The ESP32, the main controller of the robotic system, uses internet through integrated Wi-Fi connectivity to receives control commands from the cloud and generates appropriate control signals to drive the motors through the motor driver.

The control command received by the ESP32, contains two parameters: direction & speed. The speed value is used to generate PWM signals using ESP32 LED PWM controller of 8-bit resolution, which provides 256 discrete levels from 0-255 for controlling the speed of motors & the direction value determines the digital output signals to control the direction of the motor rotation through motor driver. Two PWM channels are used to control the speed of the left and right motors independently.

The algorithm implemented in the ESP32 also ensures safe switching between directions. When the direction changes (for example, forward to backward), the ESP32 first stops the motors for an instant before activating the new direction which reduces mechanical stress on motors, which improves the reliability and lifespan of the hardware components.

Additionally, the ESP32 keeps track of the previous direction of motion. If the received command has the same direction as the current motion, only the PWM duty cycle is updated to adjust the motor speed without resetting the direction control signals this is useful when acceleration happens in forward & backward direction.

L298N Motor Driver

The L298N motor driver module is used to control the speed & direction of DC motors of the robot. The module receives direction control signals from the ESP32 through pins IN1, IN2 for motor 1 & IN3, IN4 for motor 2, while the ENA and ENB pins receive PWM signals for controlling the speed of motors. Based on these signals, the motor driver controls the direction and speed of the robot's wheels.

4 Experimental Results

To evaluate the system, we conducted several experiments. The robot was controlled through an online web interface, with control commands sent via Firebase Realtime Database or the HiveMQ MQTT broker.

During the experiments, control commands such as forward, backward, left, right, and stop were sent from the web interface and received by the ESP32 through cloud which is successfully parsed and corresponding PWM & digital signals for controlling the motors through motor driver are generated.

The average command latency observed between the web interface and robot response was 300–450ms approx. when using Firebase Realtime Database & 200–350ms approx. in case of HiveMQ MQTT broker due to its lightweight publish–subscribe MQTT-based communication model.

Additionally, the PWM-based speed control mechanism provided smooth acceleration of the motors, when the control button was held down. The direction switching algorithm ensures safe motor operation by momentarily stopping the motors before reversing direction.

Table 1. Experimental Results Table

Parameter	Specification / Result
Control Interface	Web Browser (HTML, CSS, JavaScript)
Microcontroller	ESP32
Communication Methods	Firebase Realtime Database / HiveMQ MQTT Broker
Average Control Latency (Firebase)	~300–450ms
Average Control Latency (MQTT)	~200–350ms
Command Format	(direction, speed)
PWM Resolution	8-bit (0–255 speed levels)
Robot Control Response	Real-time response achieved
Speed Control	Smooth acceleration using PWM
Network Connectivity	Stable Wi-Fi communication

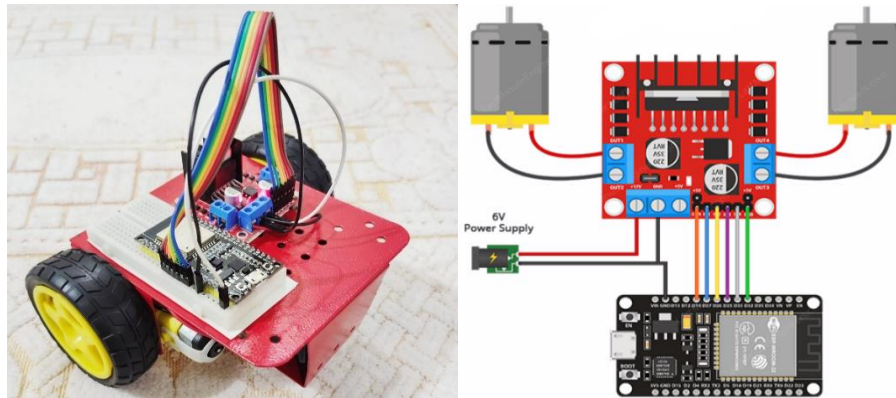


Fig. 9. Implemented Internet Controlled Bot & Circuit Diagram

5 Future Work

5.1 4G / 5G GSM Communication Module Integration

The current internet-controlled robot relies on Wi-Fi communication, which limits operation to areas with available Wi-Fi networks. In future work, a 4G/5G GSM module (e.g., SIM800 or SIM7600) can be integrated with the ESP32 to enable cellular persistent network communication. This would allow the robot to operate in remote locations where Wi-Fi infrastructure is not available with wider coverage.

5.2 Video Streaming Integration

A camera module such as ESP32-CAM can be integrated to provide live video streaming, allowing users to visually monitor the robot while controlling it remotely.

5.3 GPS Module Integration

A GPS module can be integrated into the system to enable real-time location tracking of the robot while it is being controlled remotely.

5.4 Accelerometer, Gyroscope & Magnetometer Integration

Integrating sensors such as an accelerometer, gyroscope, and magnetometer would enable the robot to measure orientation, acceleration, and heading (direction). This would allow more accurate motion tracking, improved navigation, and better stability control during movement.

5.5 Rescue Bot or Drone

The proposed Internet-controlled robot can be extended into a rescue bot or drone by integrating environmental monitoring sensors such as temperature, gas, humidity, etc. with above first 4 integrations. In emergency situations, a person could send a distress message through a publicly accessible interface or phone. The GPS location of that message is automatically transmitted to the robot or drone. Using GPS and onboard sensors, the robot automatically navigates toward the target location, collect & analyze the surrounding environmental conditions and generate an initial situation report. This feature is especially useful in disaster areas where rescue teams can't quickly understand the seriousness of the situation.

5.6 Multi-Robot Control

The system can be extended to support the control of multiple robots simultaneously using different topics (in HiveMQ) or nodes (in Firebase Realtime Database).

6 Conclusion

The system presents a simple and low-cost way to control a robot remotely using ESP32 and cloud technologies. Users can control the robot through a web interface, with commands sent via Firebase or MQTT. The ESP32 processes these commands in real time and controls the motors accordingly. Overall, the system works reliably, with both cloud services HiveMQ MQTT broker & Firebase Realtime Database.

References

1. Malaga, A.K., Ch, R.: Internet controlled robots. *International Journal of Engineering Research and Development* **4**(11), 47–50 (2012)
2. Kadir, W.M.H.W., Samin, R.E., Ibrahim, B.S.K.: Internet controlled robotic arm. In: *Procedia Engineering*, vol. 41, pp. 1065–1071. Elsevier (2012). <https://doi.org/10.1016/j.proeng.2012.07.284>
3. Kiranmai, K., Kishore Kumar, M.: Design and implementation of supervising robot by using SMS. *International Journal of Advanced Research in Electrical, Electronics and Instrumentation Engineering* **2**(11), (2013)
4. Fahmidur, R.K., Munaim, H.M., Tanvir, S.M., Sayem, A.S.: Internet controlled robot: A simple approach. In: *2016 International Conference on Electrical, Electronics and Optimization Techniques (ICEEOT)*, pp. 1190–1194 (2016)
5. Raut, A.P., Raj, A., Patil, S., Katkar, A.: Internet controlled techrobot using Raspberry Pi. *International Journal of Engineering Research & Technology (IJERT)* **9**(8) (2020)
6. Kabir, H., Tham, M.-L., Chang, Y.C.: Internet of robotic things for mobile robots: Concepts, technologies, challenges, applications, and future directions. (2023)
7. Kanna, R.K., Panigrahi, B.S., Sucharita, S., Pravallika, B., Sahoo, S.K., Gupta, P.: Real-time remote-controlled human manipulation medical robot using IoT module. (2024)
8. Emmi, L., Fernández, R., Gonzalez-de-Santos, P., Francia, M., Golfarelli, M., Vitali, G., Sandmann, H., Hustedt, M., Wollweber, M.: Exploiting the internet resources for autonomous robots in agriculture. *Agriculture* **13**(5), 1005 (2023).